

Régressions en SAS : PROC REG, LOGISTIC et MACROS

Luis Alberto Gomez

March 17, 2026

Introduction

- SAS propose plusieurs outils pour analyser et automatiser les régressions.

Procédures clés

- **PROC REG** : régression linéaire (OLS)
- **PROC LOGISTIC** : logit / probit

Automatisation

- **Macros SAS** : automatisation et génération de code

PROC REG : Régression linéaire (OLS)

Syntaxe de base

```
proc reg data=dataset;  
    model y = x1 x2 x3 / options;  
run;  
quit;
```

Options clés

- SELECTION= : STEPWISE, FORWARD, BACKWARD
- OUTPUT= : sauvegarde des résidus, prédictions, valeurs ajustées

PROC REG : Exemple de base

```
proc reg data=dataset;  
    model y = x1 x2 x3;  
run;  
quit;
```

PROC REG : Exemple avec options

```
proc reg data=mydata;  
    model y = x1 x2 x3 / clb vif dw;  
    selection stepwise;  
run;  
quit;
```

Syntaxe de base

```
proc logistic data=dataset;  
    model y(event='1') = x1 x2 x3 / options;  
run;
```

Options clés

- LINK= : logit, probit, cloglog
- OUTPUT= : sauvegarde des prédictions, résidus, probabilités
- SELECTION= : sélection automatique de variables
- RSQUARE : pseudo R^2

PROC LOGISTIC : Exemple Logit de base

```
proc logistic data=dataset;  
    model y(event='1') = x1 x2 x3;  
run;
```

PROC LOGISTIC : Exemple Logit avec options

```
proc logistic data=mydata;  
    model y(event='1') = x1 x2 x3 / link=logit rsquare  
    ;  
    selection stepwise;  
    output out=pred p=predicted;  
run;
```


Syntaxe

```
proc logistic data=dataset;  
    model y(event='1') = x1 x2 x3 / link=probit;  
run;
```

- Utilisé si la variable latente suit une distribution normale
- Fournit coefficients, tests de Wald, pseudo R^2

PROC LOGISTIC : Exemple Probit

```
proc logistic data=dataset;  
    model y(event='1') = x1 x2 x3 / link=probit;  
run;
```

ODS OUTPUT : ParameterEstimates

Objectif Récupérer les coefficients estimés dans un dataset SAS

Syntaxe

```
ods output ParameterEstimates=betas;  
  
proc reg data=mydata;  
    model y = x1 x2 x3;  
run;  
  
ods output close;
```

Résultat

- Table betas contenant :
 - Coefficients estimés
 - Erreurs standards
 - Statistiques t et p-values

Option OUTEST

Objectif Créer un dataset contenant les coefficients estimés directement depuis la procédure

Syntaxe

```
proc reg data=mydata outest=betas;  
    model y = x1 x2 x3;  
run;
```

Caractéristiques

- Une ligne par modèle estimé
- Colonnes = coefficients (x1, x2, etc.)
- Peut inclure :
 - `_RSQ_` : R^2
 - `_ADJRSQ_` : R^2 ajusté

Différence entre ODS OUTPUT et OUTEST

- Lorsqu'on récupère les résultats d'une procédure SAS, deux méthodes principales existent :
 - **ODS OUTPUT** : capture des tables de sortie détaillées
 - **OUTEST** : option de la procédure pour enregistrer les coefficients

Différence clé

- **ODS OUTPUT** : très détaillé, inclut tests, erreurs standards, statistiques t et p-values
- **OUTEST** : plus compact, ligne par modèle, pratique pour stockage rapide et automatisation

Introduction aux macros SAS

- Les macros permettent d'automatiser et de rendre le code dynamique.
- Deux éléments principaux :
 - **Macro variables** : stockage de valeurs
 - **Macro programmes** : génération de code SAS
- Utilisées pour :
 - Répéter des analyses
 - Paramétrer des modèles
 - Générer du code automatiquement

Macro variables

Définition

Une macro variable stocke une valeur texte utilisable dans le code.

Syntaxe

```
\%let var = valeur;
```

Utilisation

```
\%let y = income;  
  
proc reg data=mydata;  
    model &y = x1 x2;  
run;
```

- & permet d'appeler la macro variable

Définition

Fonctions utilisées dans les macros pour manipuler du texte ou des valeurs.

Exemples

```
\%let x = 10;  
\%let y = %eval(&x + 5);  
  
\%let name_maj = %upcase(name);
```

- %eval() : opérations arithmétiques
- %upcase() : transforme en majuscules
- %substr() : extrait une sous-chaîne

Macro programmes

Définition

Permet de créer une procédure réutilisable.

Syntaxe

```
%macro regression(y, x);  
proc reg data=mydata;  
    model &y = &x;  
run;  
%mend;
```

Appel

```
%regression(y=income, x=x1 x2 x3);
```

Exemple avec boucle

```
%macro reg_macro;  
%do i = 1 %to 3;  
  
proc reg data=mydata;  
    model y = x&i;  
run;  
  
%end;  
%mend;  
  
%reg_macro;
```

- Génère plusieurs régressions automatiquement
- Très utile pour simulations ou tests multiples

Exemple complet

```
\%macro regression(depvar);  
  
proc logistic data=mydata;  
    model &depvar(event='1') = x1 x2 x3;  
run;  
  
\%mend;  
  
\%regression(y1);  
\%regression(y2);
```

- Réutilisation du même modèle pour plusieurs variables dépendantes
- Gain de temps et réduction des erreurs